CAROM Experiments 2 and 3: Detailed Experiment and Results Summary

Date: 2026-07-21
Project: CAROM Execution Semantics
Scope: One-seed TinyLM mechanism screens; Exp2 compiled-channel execution and Exp3 teach-loop operator acquisition

Executive summary

Experiments 2 and 3 each validate one local component of the broader CAROM proposal, but neither validates the end-to-end ambitious claim.

Experiment 2 shows that a language-conditioned pairwise scorer can learn substantial information about a program's dependency graph and drive a CAROM core to moderate endpoint accuracy on program lengths seen during training. Its final logged L=2--4 task accuracy was 0.695 and edge accuracy was 0.865. However, held-out L=5 task accuracy was only 0.306, and itinerary Kendall tau declined from an early peak of 0.990 to 0.492 while endpoint accuracy improved. Thus it does not yet demonstrate length-generalizing compilation or that the intended ordered heteroclinic itinerary is the mechanism producing the answer.

Experiment 3 shows that two fresh operator slots can be trained from synthetic input/output examples while the old operator library remains frozen. Both new skills reached 0.632 mixed-program accuracy when invoked using their taught names, close to the 0.659 old-only baseline. Transfer to description variants not used by the registration loss was weaker and skill-dependent: 0.410 for `shl` and 0.275 for `sq`. Held-out command representations also produced a measurable novelty signal before teaching. The observed old-task change from 0.702 to 0.627 is not a valid causal interference estimate because the two evaluations used different random examples.

The combined result is a credible toy-scale proof of principle for supervised graph extraction and slot-level operator acquisition. The unresolved parts are the central scientific ones: robust semantic routing, causal ordered execution, structural extrapolation, and behavior-preserving library extension.

Evidence, provenance, and epistemic status

This report distinguishes four evidence levels:

- **Observed:** present in the retrieved raw logs or inspected source.
- **Supplied:** reported in Ben's accompanying sandbox report but not produced by the independently retrieved run.
- **Inferred:** a reasoned interpretation of observed measurements.
- **Not established:** a claim that the current design cannot support.

The runs used seed 0 and a three-layer, width-96 TinyLM. Although Exp2 ran on a GPU-equipped RunPod host, its completed artifact is explicitly the TinyLM arm, not GPT-2. A separate GPT-2 arm had been started earlier, but no completed GPT-2 result artifact was recovered. Exp3 likewise used TinyLM and a synthetic teacher, not an external LLM.

The exact launch commands and a complete remote environment manifest were not captured. Effective configurations below are reconstructed from source defaults and log output. This is sufficient to interpret

the measurements but not a gold-standard independent reproduction package.

Experiment 2: compiled heteroclinic channel

Scientific question

Can dependency language be converted into a per-instance heteroclinic control topology that executes a scrambled sequence of compositional operations in the correct order, including at a program length not seen during training?

The conceptual decomposition is:

1. encode each natural-language command span with a frozen language model;
2. infer the directed predecessor/successor graph between spans;
3. compile that graph into a generalized Lotka--Volterra inhibition matrix;
4. use the resulting activity itinerary to mix reusable neural operators;
5. execute the program over a six-symbol workspace.

Synthetic task

Each example contains a vector of six values in Z8 and a program of distinct primitive transformations drawn from `inc`, `dbl`, `neg`, `rev`, `shr`, `swp`, and `cms`. Each primitive has three English paraphrases. Programs are presented in scrambled order rather than execution order. The first command says “to begin”; each later command contains an “after you ...” clause referring to its predecessor. Distinct primitives make references unambiguous.

Training samples program lengths L=2--4. Evaluation includes fresh L=2--4 examples and a structural holdout at L=5. Chance per-output-symbol accuracy is 1/8 = 0.125.

Model

The completed run used:

- TinyLM: three causal-transformer layers, hidden size 96, four attention heads;
- 1,500 steps of next-token pretraining on the same procedurally generated command language, after which TinyLM was frozen;
- mean-pooled frozen span representations for each command;
- a pairwise MLP edge scorer over `[h_i, h_j, h_i * h_j]`;
- a learned entry scorer selecting the initial active command;
- a compiler mapping predicted edges to fixed GLV inhibition constants;
- eight shared neural operators over a 48-dimensional, six-slot workspace;
- 72 GLV/update integration steps per example.

The graph compiler assigns low successor inhibition and high reverse-direction suppression for a predicted edge. Command activity weights are then used to mix the shared operator bank at every integration step.

Training protocol

The effective configuration was seed 0, 4,000 optimizer updates, batch size 128, and edge-supervision weight 1.0. Training combined:

- endpoint cross-entropy on the transformed six-symbol output;
- binary cross-entropy on direct predecessor edges; and
- a lightly weighted supervised entry-command loss.

The run therefore tests the compiler-supervised regime only. It does not contain the task-only (`edges=0`), oracle-topology, null-topology, shuffled-edge, or non-heteroclinic controls needed to isolate causal contributions.

At each 200-step logging point, evaluation generated new random examples using the same RNG object as training. Consequently, checkpoints are not evaluated on a fixed paired corpus, and evaluation calls alter the subsequent training sample stream.

Metrics

- **Task accuracy:** fraction of the six output symbols predicted correctly.
- **Edge accuracy:** thresholded accuracy over all live off-diagonal command pairs. This is class-imbalanced; only L-1 of L(L-1) directed pairs are positive for a length-L chain. Under the approximately uniform L=2--4 mix, an all-negative classifier is expected to score about 0.70.
- **Itinerary Kendall tau:** Kendall ordering of the sequence obtained from the argmax active command across 72 integration steps, after collapsing adjacent repeats.

The tau implementation does not penalize omitted commands: an ordered subset can receive a high score. It also lacks phase-coverage, transition-recall, dwell-time, and exact-itinerary checks. Tau should therefore be treated as a screening diagnostic rather than a validated itinerary measure.

Results

TinyLM next-token loss fell from 4.748 at step 0 to 0.363 at step 1,200. The compiled CAROM learning curve was:

Update	L=2--4 task acc.	Edge acc.	L=2--4 tau	L=5 task acc.	L=5 tau
0	0.116	0.626	-0.195	0.095	-0.146
400	0.237	0.755	0.990	0.147	0.964
1,000	0.437	0.797	0.839	0.197	0.842
2,000	0.607	0.822	0.537	0.232	0.410
2,600	0.675	0.850	0.567	0.331	0.412
3,000	0.706	0.860	0.474	0.346	0.341
3,600	**0.741**	0.866	0.531	**0.350**	0.367
3,800	0.695	0.865	0.492	0.306	0.387

The checkpoint was saved at update 3,999, but no evaluation was logged after 3,800. Because every row uses a new random evaluation set, small reversals should not be interpreted as precise within-run regressions.

Interpretation

Observed: edge accuracy improved by 0.239 absolute, from 0.626 to 0.865. Relative to the approximate 0.70 all-negative baseline, this is nontrivial but less dramatic than the raw number suggests. The run does show that frozen span representations contain information usable by the supervised edge scorer.

Observed: in-distribution task accuracy rose far above chance, reaching 0.695 at the final logged point and 0.741 at its best logged point. The compiled system therefore learned useful short-program computation.

Observed: L=5 accuracy was consistently much lower, ending at 0.306 versus 0.695 for L=2--4. It exceeded chance but did not preserve the trained-length performance level. This is partial transfer, not structural generalization.

Observed: tau became almost perfect early, then declined as endpoint task accuracy continued to increase. One possible interpretation is mechanistic drift: the model increasingly solves endpoints through distributed operator mixing or workspace dynamics rather than a clean one-command-at-a-time channel. An equally important alternative is metric failure: early near-perfect tau may reflect an ordered subset without complete itinerary coverage.

Not established: that language compiled the graph in a pretrained semantic sense. TinyLM learned the same small synthetic vocabulary and template family used by the task. No lexical, syntactic, or semantic out-of-distribution split was used.

Not established: that the graph compiler caused endpoint performance. Edge supervision, entry supervision, task training, fixed GLV constants, and the operator core changed together, without causal ablations.

Exp2 conclusion

Exp2 is a positive mechanism screen for learnable dependency-edge extraction and a negative result for the stronger length-generalizing, causally itinerant claim. Its most scientifically useful signal is the divergence between edge accuracy, endpoint accuracy, and itinerary tau: these are different properties and cannot be substituted for one another.

Experiment 3: teach-loop operator acquisition

Scientific question

Can a frozen language representation anchor a newly registered neural operator that is learned from teacher-generated examples, while preserving the existing operator library and permitting invocation through alternative descriptions?

Synthetic task and holdout structure

Exp3 extends the Z8 six-slot task to nine primitives. Seven old primitives are used to train the original CAROM cap. Two are held out from cap training:

- `shl`: rotate the vector left by one position;
- `sq`: square every value modulo 8.

Each primitive has three paraphrases. TinyLM pretraining includes all nine skills, all paraphrase variants, and sequences containing the input and output of composed programs. Thus `shl` and `sq` are held out from the operator cap, but not from the representation model's language or functional pretraining.

Model and teaching mechanism

The frozen base is again a three-layer, width-96 TinyLM. The original cap has eight shared operators and a language-conditioned routing hypernetwork. A new skill adds:

- a fresh single-operator neural core;

- a frozen 96-dimensional description anchor;
- a trainable cosine-routing scale and bias.

The anchor is the average TinyLM span representation of paraphrase variant 0 in several program contexts. For each new skill, the synthetic teacher supplies 1,024 exact input/output pairs with zero label noise. Teaching runs 400 updates. Only the new operator and its routing scale/bias are trained; the base LM, existing operator bank, old router, embeddings, and output head remain frozen.

The loss combines forced-new-slot operator fitting with full-softmax routing calibration. Positive examples require the taught-name representation to select the new slot; fresh old-command negatives are penalized for selecting it.

Stages and effective protocol

All stages used seed 0:

1. **LM:** 800 updates over programs containing all nine skills and all three paraphrase variants.
2. **Cap:** 800 updates on programs using only the seven old skills and paraphrase variants 0 and 1.
3. **Teach:** sequential registration of `shl` and `sq`, 1,024 noiseless pairs and 400 updates per skill, using variant 0 as the anchor/taught name.
4. **Evaluate:** 384 fresh mixed programs per call. The new skill is forced into the first program position; remaining positions use old skills.

No sample-efficiency or teacher-noise sweep was executed, despite M3 being part of the experiment concept.

Metrics

- **M1 acquisition:** mixed-program output accuracy when the first operation is a new skill, invoked either through its taught name (variant 0) or variants 1 and 2.
- **M2 interference:** old-only program accuracy before and after registration. In this implementation these calls draw different random batches, so M2 is not paired and does not identify a causal change.
- **M3 sample efficiency:** intended accuracy-versus-example-count/noise study; not run.
- **M4 novelty:** before registration, the old router's mean maximum routing probability and entropy for old versus held-out command representations.

Training dynamics

TinyLM next-token loss decreased from 4.822 to 1.091 by update 600. Old-cap mixed-program accuracy rose from 0.139 at cap update 0 to 0.738 at update 600 and was 0.707 at update 750. The saved cap completed update 799.

Direct new-operator pair losses were:

Skill	update 0	update 100	update 200	update 300
`shl`	7.398	2.072	0.055	0.014
`sq`	5.022	0.001	0.001	0.000

This shows successful fitting of the two deterministic operators under forced routing. It does not by itself test whether language routes to those operators.

Evaluation results

Evaluation	`shl`	`sq`	Reference
Taught name, mixed programs	0.632	0.632	old-only baseline 0.659
Alternative paraphrases, mixed programs	0.410	0.275	chance 0.125

Pre-teaching novelty measurements were:

Command class	mean max old-router probability	mean routing entropy
old skills	0.942	0.224
held-out `shl`/`sq`	0.835	0.438

Old-only accuracy was 0.702 on the pre-registration sample and 0.627 on the post-registration sample. These samples were independently generated and were separated by all teaching RNG consumption. The 0.075 difference is therefore a warning signal, not a measured interference effect.

Comparison with the supplied sandbox report

Metric	supplied sandbox	retrieved run
`shl`, taught name	0.601	0.632
`sq`, taught name	0.584	0.632
`shl`, alternative paraphrases	0.408	0.410
`sq`, alternative paraphrases	0.438	0.275
old-only baseline	0.691	0.659

The `shl` paraphrase score reproduced closely, while `sq` differed by 0.163. Because both results are single-seed and use fresh finite evaluation samples, this disagreement demonstrates that the paraphrase-transfer conclusion is not yet stable. The novelty direction replicated qualitatively: held-out commands had lower maximum routing confidence and higher entropy than old commands.

Interpretation

Observed: fresh neural slots can acquire the two deterministic operators without modifying the old operator parameters. Both operators also function in multi-step programs when invoked by the exact registered phrase.

Observed: alternative-description transfer is above chance but markedly below exact-name invocation, especially for `sq`. Registration therefore has some representation-level generalization but not robust semantic routing.

Observed: the old router is less confident and more entropic on held-out command representations. This supports novelty detection as a promising signal, but no ROC curve, held-out threshold, or false-positive

calibration was run.

Inferred: adding logits to a shared softmax can alter old behavior even when old weights are frozen. This is the correct mechanism to investigate for interference. The current unpaired M2 values do not quantify that effect.

Not established: LLM teaching. TinyLM was trained directly on all new-skill descriptions and functional examples; the "teacher" was an exact Python function generating noiseless pairs. There was no prompted model, natural instruction, ambiguous specification, program synthesis, or teacher error.

Not established: arbitrary compositional insertion. Evaluation always puts the new skill first, so it does not test new-skill routing at middle or final positions, repeated use, or interaction between multiple newly taught skills.

Exp3 conclusion

Exp3 validates extensible operator fitting plus an anchor-based routing interface at toy scale. Its strongest result is exact-name acquisition near the old-task baseline. Its main negative result is weak and unstable paraphrase transfer. Its most important methodological lesson is that frozen weights are not equivalent to frozen behavior when a shared routing action space expands.

Combined scientific interpretation

The two experiments address complementary halves of a prospective CAROM learning system:

Capability	Exp2	Exp3	Current status
extract control structure from descriptions	supervised pairwise edges	novelty/anchor routing	promising toy signal
execute known operators	compiled GLV mixing	scheduled old cap	works on trained distributions
acquire a new operator	not tested	new slot from I/O pairs	demonstrated with exact teacher
semantic description transfer	template-bound TinyLM	paraphrase test	weak/unstable
structural extrapolation	L=5 holdout	not tested	poor in Exp2
causal heteroclinic execution	tau diagnostic only	scheduled, not GLV-compiled	not established
noninterfering extension	not tested	unpaired warning signal	not established

The natural synthesis is not yet "language compiles and teaches a modular heteroclinic computer." A defensible statement is narrower:

> In a small synthetic domain, frozen learned text representations support supervised dependency prediction and anchor-based registration of newly fitted neural operators. Current implementations do not yet generalize reliably across program length or paraphrase, and their causal execution and noninterference properties remain unvalidated.

Recommended next experiments

Priority 0: repair the instruments

1. Freeze deterministic train, validation, and test corpora with independent RNG streams and shared example IDs across all arms.
2. Validate itinerary measurements on constructed positive and negative trajectories. Add station coverage, correct-transition precision/recall, exact-order rate, dwell distributions, revisitation count, and endpoint sensitivity to trajectory swapping.
3. Replace threshold edge accuracy with AUROC, AUPRC, positive recall, exact-graph accuracy, and calibration curves.
4. Make Exp3 M2 paired: evaluate identical old programs before and after each slot registration and decompose errors into routing changes versus operator changes.

Priority 1: isolate causal components

For Exp2, compare at least five seeds across compiler-supervised, task-only, oracle graph, null graph, shuffled graph, and a non-heteroclinic scheduled executor. Evaluate both update-matched and wall-clock-matched views.

For Exp3, use forced-oracle routing to separate operator acquisition from description routing. Compare shared-softmax registration with a gated novelty router and with a KL/no-regression penalty on frozen old examples.

Priority 2: test the intended generalizations

For Exp2, use a genuinely frozen GPT-2 adapter, held-out paraphrase families, dependency syntax changes, lexical substitutions, and lengths beyond L=5.

For Exp3, sweep teacher examples and label-noise rates, register skills in multiple orders, evaluate the new skill in every program position, teach several skills sequentially, and test descriptions not seen during base pretraining. Only after these controls should the synthetic teacher be replaced by prompted LLM-generated specifications.

Artifact manifest

Artifact	SHA-256
Exp2 local run-source copy	`9171bcd065226d3f5ca96d7bb8f9f20d9f144a11f251923382cbefb7be112223`
Exp2 originally supplied source	`e8acb6a3dd9fb761c76487bfcd1eb49a4c93ebd2540f438a50e73550285d52b8`
Exp2 raw log	`89020404fbde00deab6aa9f0ff0f52a94cc927df4b83abbecafc14ba2dbae9bd`
Exp3 source	`e8babc238d6835256aa224e381285485d9146d984e6ae660e76ccb849918a6ff`
Exp3	`b29d26ebdaefa334a5e103d63a03911ded9cc662cd92cdabdc02ca165d59dbf6`

Artifact	SHA-256
supplied report	
Exp3 LM log	`58340b51d48c4810692956464ba3a07eb9ce6fa448d338a81adae0fe4aa9af9a`
Exp3 cap log	`9806614fdfd8c0c8646a1390348feeffb1849c3a104c26e10472ecfe4f656a63`
Exp3 teaching log	`041b1edab7043ab5b15c0343087723fe04e1065ebe52db7cb0a08e3f26b0e96c`
Exp3 evaluation log	`68c429668730a3a430eedeac89c6602bf6b0e62587b206688ea8a6293d521f26`

Source ledgers:

- `experiments/20260721T055900Z-exp2-compiled-channel-tinylm/RUN.md`
- `experiments/20260720T134700Z-exp3-teach-loop/RUN.md`